CampusPass

Architecture and Database Schema Design

College event discovery and seat booking | Version 1.0 | 28 October 2026

1. Project overview

College events are announced on WhatsApp and booked through Google Forms, so seats get overbooked, students miss events and organisers do not know who is coming. CampusPass replaces that with one web application: students browse approved events and book one seat each, organisers publish events and see who booked, and admins approve events and manage roles.

Users and roles

Role	Goal
Visitor (not logged in)	Browse and search approved events.
Student	Book a seat, cancel own booking, review an event after it ends.
Organiser	Create and edit own events, see who booked them.
Admin	Approve or reject events, change user roles.

Business rules the design must enforce

- Only **approved** events are public. A new event stays **pending** (hidden) until an admin approves it.
- One active seat per student per event, and an event never goes over its capacity.
- A student sees and cancels only their own bookings, and only up to 2 hours before the event starts.
- Organisers edit only their own events.
- The API refuses anything a role is not allowed to do. Hiding a button in the UI is never the protection.

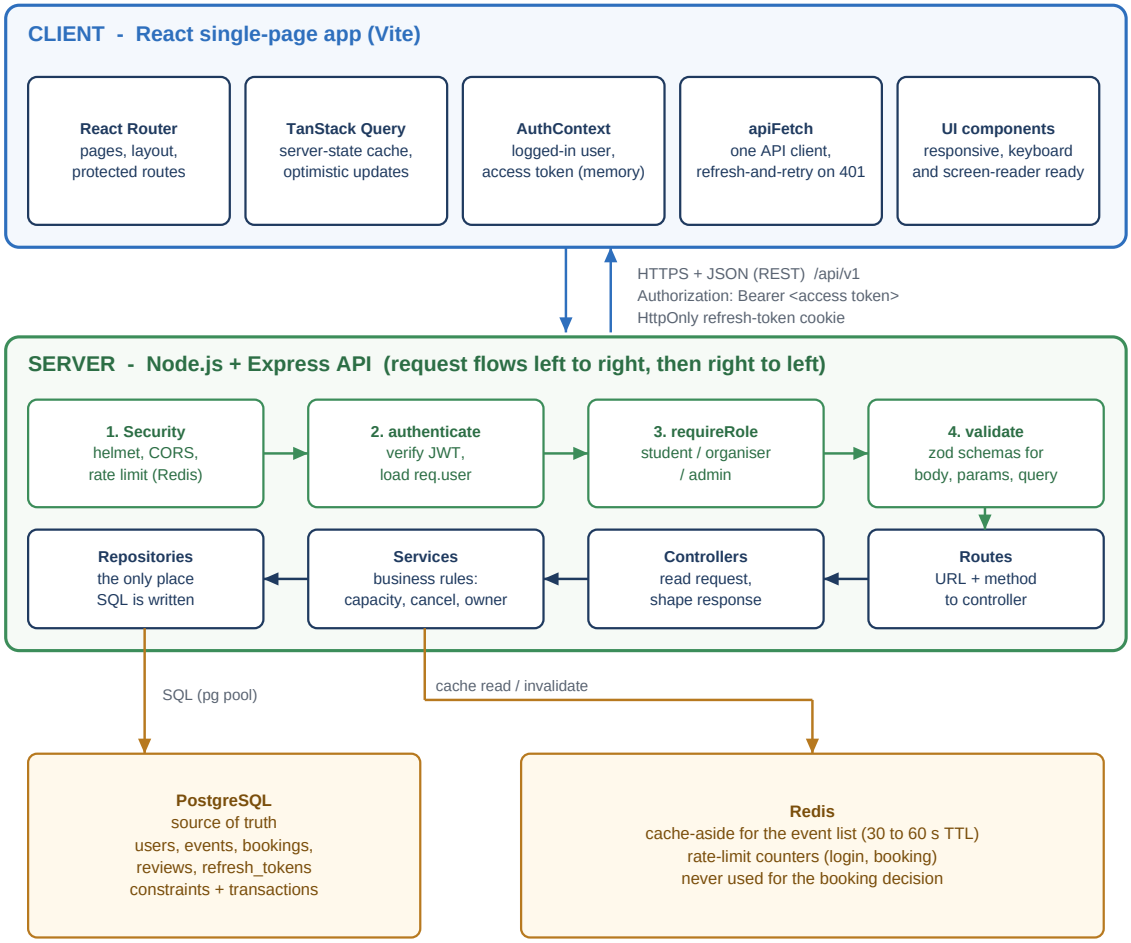
Scope

Must have	Nice to have
Browse upcoming events as cards, with search and category filter Event detail page with seats left Register, log in, stay logged in Book a seat with clear failure messages (already booked, full, too many tries) My bookings page with cancel Organiser creates and edits own events	Organiser sees the attendee list Admin approves pending events Review an event after it ends

Technology stack

Layer	Choice	Why
Front end	React (Vite), React Router, TanStack Query, plain CSS (mobile first)	Component model, server-state caching, URL-based filters
Back end	Node.js, Express, zod, JWT, bcrypt	REST API with layered structure and input validation
Database	PostgreSQL	Transactions and constraints protect capacity and uniqueness
Cache	Redis	Event-list cache and rate-limit counters
Quality	Lighthouse, axe DevTools, GitHub	Accessibility and performance targets, version control

2. System architecture



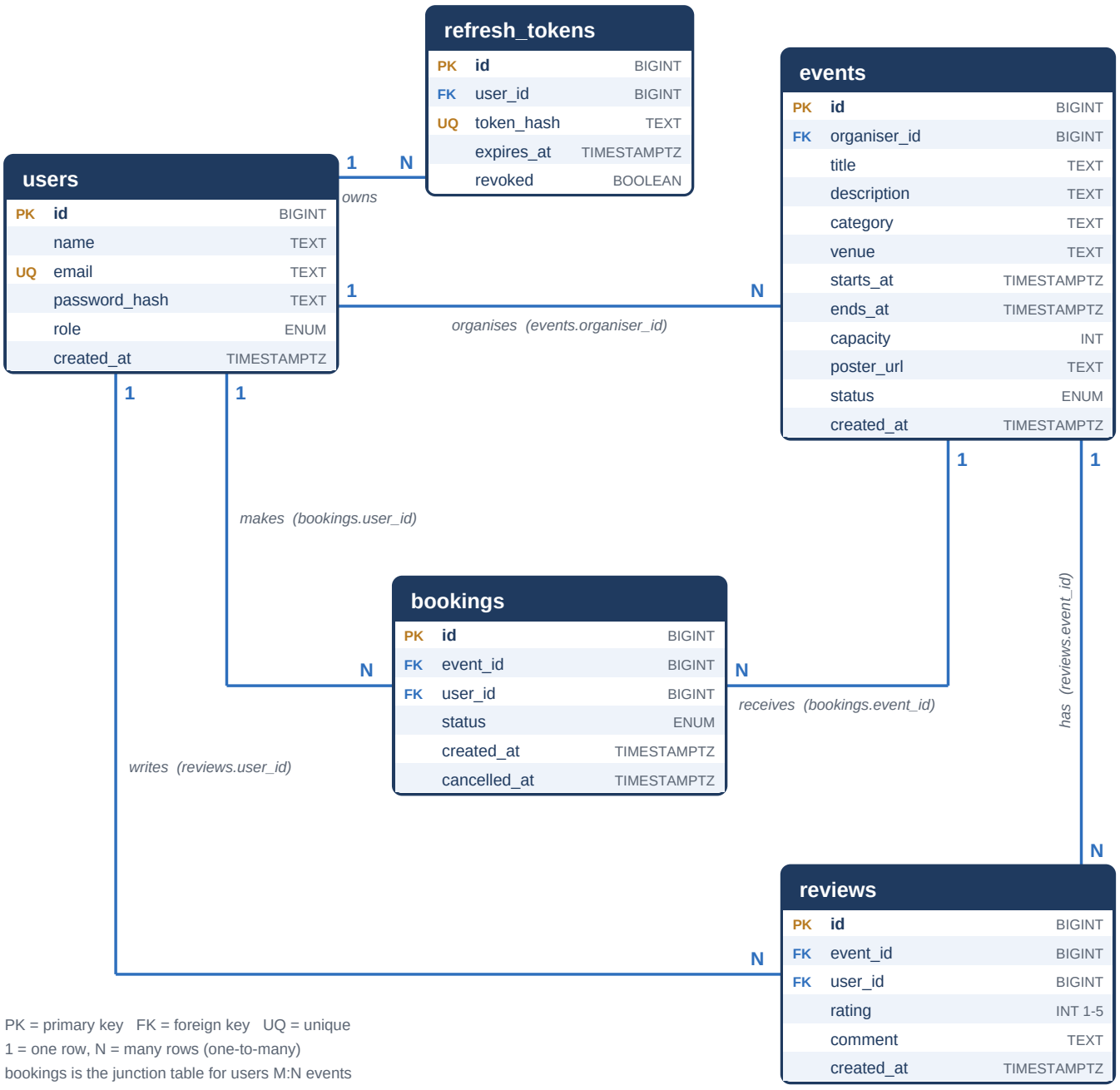
Front-end responsibilities

Concern	Decision
Server data (events, seats left, my bookings)	TanStack Query with query keys such as ['events', filters] and ['me','bookings']. Invalidate after every write so the screen never shows stale seats.
Shareable state (search text, category, page)	Stored in the URL query string, so refresh, Back and shared links all work.
Session (user, access token)	AuthContext. Access token lives in memory only; the refresh token is an HttpOnly cookie. On app start one call to /auth/refresh restores the session.
Local UI state (dialog open, form inputs)	useState in the lowest component that needs it.
Pages	Events list, event detail, login and register, my bookings, organiser dashboard (lazy loaded), admin dashboard (lazy loaded), not found.

Back-end responsibilities

Layer	Responsibility
Middleware	Security headers and CORS, Redis rate limiting, JWT authentication, role check, zod validation. A central error handler (last middleware) turns any AppError into { error: { code, message } } with the right HTTP status.
Routes and controllers	Map URL and method to a handler; read the request; return { data } or { error }. No business logic.
Services	All rules: capacity, one seat per student, 2-hour cancel window, ownership, approval workflow.
Repositories	The only code that writes SQL. Parameterised queries only. Booking runs inside a transaction.

3. Entity-relationship diagram



PK = primary key FK = foreign key UQ = unique
1 = one row, N = many rows (one-to-many)
bookings is the junction table for users M:N events

Relationships

Relationship	Cardinality	Foreign key	Meaning
users - events	1 : N	`events.organiser_id`	One organiser owns many events; each event has exactly one organiser.
users - bookings	1 : N	`bookings.user_id`	A student can hold many bookings (one per event).
events - bookings	1 : N	`bookings.event_id`	An event receives many bookings, limited by capacity.
users - events (via bookings)	M : N	`bookings table`	Students attend many events and events host many students; bookings resolves it.
users - reviews	1 : N	`reviews.user_id`	A student can write many reviews, one per event.
events - reviews	1 : N	`reviews.event_id`	An event can have many reviews.
users - refresh_tokens	1 : N	`refresh_tokens.user_id`	A user can be logged in on many devices.

4. Table definitions (PostgreSQL)

Enum types: **user_role** ('student','organiser','admin'), **event_status** ('pending','approved','rejected'), **booking_status** ('booked','cancelled').

users

Column	Type	Constraints and notes
id	BIGSERIAL	PRIMARY KEY
name	TEXT	NOT NULL
email	TEXT	NOT NULL, UNIQUE (used for login)
password_hash	TEXT	NOT NULL. bcrypt or argon2 hash, never the password.
role	user_role	NOT NULL, DEFAULT 'student'
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

events

Column	Type	Constraints and notes
id	BIGSERIAL	PRIMARY KEY
organiser_id	BIGINT	NOT NULL, FOREIGN KEY to users(id)
title, description, category, venue	TEXT	NOT NULL
starts_at, ends_at	TIMESTAMPTZ	NOT NULL, CHECK (ends_at > starts_at)
capacity	INT	NOT NULL, CHECK (capacity > 0)
poster_url	TEXT	Optional
status	event_status	NOT NULL, DEFAULT 'pending'. Public only when 'approved'.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

bookings

Column	Type	Constraints and notes
id	BIGSERIAL	PRIMARY KEY
event_id	BIGINT	NOT NULL, FOREIGN KEY to events(id)
user_id	BIGINT	NOT NULL, FOREIGN KEY to users(id)
status	booking_status	NOT NULL, DEFAULT 'booked'. Cancel sets 'cancelled' so history is kept.
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()
cancelled_at	TIMESTAMPTZ	NULL until cancelled

reviews (nice to have)

Column	Type	Constraints and notes
id	BIGSERIAL	PRIMARY KEY
event_id, user_id	BIGINT	NOT NULL, FOREIGN KEYS to events(id) and users(id). UNIQUE (event_id, user_id).
rating	INT	NOT NULL, CHECK (rating BETWEEN 1 AND 5)
comment	TEXT	Optional
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()

Service rule: a review is allowed only after the event has ended and only if the user has a booking for it.

refresh_tokens

Column	Type	Constraints and notes
id	BIGSERIAL	PRIMARY KEY
user_id	BIGINT	NOT NULL, FOREIGN KEY to users(id)
token_hash	TEXT	NOT NULL, UNIQUE. Only the hash is stored.
expires_at	TIMESTAMPTZ	NOT NULL
revoked	BOOLEAN	NOT NULL, DEFAULT false. Set on logout or rotation.

Integrity rules and indexes

Item	Definition	Purpose
one_active_booking	`UNIQUE INDEX ON bookings(event_id, user_id) WHERE status = 'booked'`	One active seat per student per event, while allowing re-booking after a cancel.
events_public_idx	`INDEX ON events(status, starts_at, id)`	Public list query and cursor pagination.
events_category_idx	`INDEX ON events(category)`	Category filter.
bookings_user_idx	`INDEX ON bookings(user_id, status)`	My bookings page.
Derived value	`seatsLeft = capacity - count(bookings WHERE status = 'booked')`	Computed on read, never stored, so it cannot drift.
Foreign keys	`Default RESTRICT (no cascading deletes)`	Booking history is never silently removed; events are rejected or hidden, not deleted.

5. REST API design (base path /api/v1)

Endpoint	Allowed roles	Purpose
`POST /auth/register`	Public	Create a student account.
`POST /auth/login`	Public	Returns access token and sets the HttpOnly refresh cookie.
`POST /auth/refresh`	Public (cookie)	Rotates the refresh token and returns a new access token.
`POST /auth/logout`	Logged in	Revokes the refresh token.
`GET /events?q&category&cursor&limit`	Public	Approved upcoming events. Returns { data, nextCursor }.
`GET /events/:id`	Public	Event detail including seatsLeft.
`POST /events`	Organiser, Admin	Create an event with status 'pending'.
`PATCH /events/:id`	Owner organiser, Admin	Edit an event.
`GET /me/events`	Organiser	Own events including pending and rejected.
`POST /events/:id/bookings`	Student	Book a seat (transaction below).
`GET /events/:id/bookings`	Owner organiser, Admin	Attendee list.
`GET /me/bookings`	Logged in	Own bookings only.
`DELETE /bookings/:id`	Booking owner	Cancel, if more than 2 hours before the event.
`POST /events/:id/reviews`	Student with a booking	Review after the event ends.
`GET /admin/events?status=pending`	Admin	Events waiting for approval.
`PATCH /admin/events/:id/status`	Admin	Approve or reject.
`PATCH /admin/users/:id/role`	Admin	Change a user's role.

Response and error format

Success: `{ data, nextCursor? }`. Failure: `{ error: { code, message } }` with a stable code the UI can switch on.

HTTP	Code	When
401	`UNAUTHENTICATED`	Missing or expired access token (client tries /auth/refresh once).
403	`FORBIDDEN`	Logged in but the role is not allowed.
404	`NOT_FOUND`	Missing resource, or someone else's booking (avoids leaking that it exists).
409	`EVENT_FULL`	No seats left.
409	`ALREADY_BOOKED`	The student already holds an active seat.
409	`CANCEL_WINDOW_CLOSED`	Less than 2 hours before the event.
422	`VALIDATION_ERROR`	Request body, params or query failed zod validation.
429	`RATE_LIMITED`	Too many login or booking attempts.

6. Role-based access control (RBAC)

Action	Visitor	Student	Organiser	Admin
Browse approved events	Yes	Yes	Yes	Yes
Book and cancel a seat	No	Yes (own)	No	No
View my bookings	No	Yes (own)	No	No
Create an event	No	No	Yes	Yes
Edit an event	No	No	Own only	Any
See attendee list	No	No	Own events	Any
Review a finished event	No	Yes (if booked)	No	No
Approve or reject events	No	No	No	Yes
Change user roles	No	No	No	Yes

Enforced twice: the UI hides what a role cannot do (user experience), and the API middleware `requireRole` plus ownership checks in the services refuse it (security).

7. Key flows

7.1 Booking a seat (prevents overbooking)

```
BEGIN;
SELECT capacity, status, starts_at FROM events WHERE id = $1 FOR UPDATE;  -- lock the event row
-- service: event must be 'approved' and in the future, else 404
SELECT count(*) FROM bookings WHERE event_id = $1 AND status = 'booked';
-- if count >= capacity: ROLLBACK and throw EVENT_FULL (409)
INSERT INTO bookings (event_id, user_id) VALUES ($1, $2);
-- unique violation on one_active_booking: ALREADY_BOOKED (409)
COMMIT;
```

- Two students taking the last seat at the same moment are serialised by the row lock: one commits, the other receives 409 EVENT_FULL.
- Client side: the Book button updates seatsLeft optimistically, rolls back on any error, then re-fetches ['events', id] and ['me','bookings'].
- Proof for the demo: fire 20 parallel booking requests at an event with 5 seats and show that exactly 5 bookings exist.

7.2 Cancelling a booking

```
UPDATE bookings SET status = 'cancelled', cancelled_at = now()
WHERE id = $1 AND user_id = $2 AND status = 'booked'
  AND now() < (SELECT starts_at FROM events WHERE id = bookings.event_id) - interval '2 hours';
-- 0 rows updated: 404 if not found or not the user's booking, 409 CANCEL_WINDOW_CLOSED if too late
```

7.3 Authentication and session

- **Login:** verify the bcrypt hash, issue a short-lived access token (about 15 minutes) returned in the response body, and a random refresh token set as an HttpOnly cookie (only its hash is stored in refresh_tokens).
- **Client:** keeps the access token in memory (not localStorage). On a 401, apiFetch calls /auth/refresh once and retries the request.
- **Refresh:** the old refresh token is revoked and a new one issued (rotation). **Logout:** revokes the token and clears the client cache.

7.4 Caching and performance

- Redis cache-aside on the public event list with a 30 to 60 second TTL, invalidated when an event is created, edited or approved.
- The booking decision never trusts the cache: the transaction always counts live rows.
- Cursor pagination on (starts_at, id), a virtualised list, debounced search and staleTime keep 5,000 events fast.

8. Security checklist

Threat	Control
Role bypass by calling the API directly	requireRole middleware plus ownership checks in services; 404 for other users' bookings.
SQL injection and bad input	Parameterised queries only (repositories); zod validation of every body, param and query.
Token theft	Short-lived access token in memory; refresh token HttpOnly, hashed in DB, rotated.
Brute force and booking spam	Redis rate limits on login and booking, returning 429.
Cross-site requests	CORS restricted to the front-end origin with credentials; helmet security headers.

9. Assumptions

- Stack assumed to be Node.js, Express, PostgreSQL and Redis, matching the course material; entities and rules are independent of this choice.
- Role decisions left open by the brief: only students book seats; admins can edit any event; organisers cannot approve their own events. Nice-to-have features are in the schema but built after all must-haves work.